auto_ml

이진 분류(0/1) 문제를 자동으로 학습·스코어링하는 사내 라이브러리. 폐쇄 환경 이관 / 운영을 전제로 모듈화·설정 주도(YAML)로 설계되었다.

구성

```
train/                      학습 실행 스크립트 (설치 없이 python 으로 실행)
├── preprocess.py           [1/3] 전처리 + (선택) 변수 선택
├── train.py                [2/3] 모델 학습 + best 선정
├── postprocess.py          [3/3] 리포트 / artifact 저장 / test 예측 export
└── run_all.py              1~3 일괄 실행
inference/                  추론 실행 스크립트
├── preprocess.py           [1/3] artifact 전처리기로 입력 변환
├── predict.py              [2/3] best 모델 예측 (score)
├── postprocess.py          [3/3] 임계값 적용 + id 결합 → Parquet 저장
└── run_all.py              1~3 일괄 실행
auto_ml/
├── config.py               설정 dataclass + YAML 로더
├── stages/                 학습 / 추론 단계 함수 (위 스크립트가 호출)
├── pipeline.py             학습 파이프라인 (Python API)
├── preprocessing/          1) 결측 → 2) 이상치 → 2.5) skew 변환 → 3) 스케일링
├── feature_selection/      Stability Selection 변수 선택
├── models/                 LGBM / XGBoost / CatBoost / ElasticNet 래퍼 +
Ensemble + Trainer
├── tuning/                 Optuna 베이지안 하이퍼파라미터 최적화
├── reporting/              HTML + PDF 리포트 (동일 내용)
├── scoring/                배치 스코어링 (Scorer / run_scoring)
├── explain/                SHAP 해석 (python -m auto_ml.explain.runner)
├── cli/                    best 교체 (python -m auto_ml.cli.set_best)
└── utils/                  io / logger / validation / categorical
examples/                   실행 샘플(sample/), 데이터 준비 예시(credit/, titanic/), 더
미 데이터 생성
configs/                    예시 설정 (example.yaml, features.csv)
```

5단계 파이프라인

1. **전처리** — 결측 → 이상치 → skew 변환 → 스케일링 순서 고정. 학습 시 통계량을 저장해 스코어링 시 동일 적용.

2. **변수 선택** (선택) — Stability Selection (Meinshausen & Bühlmann, 2010). 전처리 직후 적용하여 안정적으로 선택되는 변수만 모델에 전달한다. `feature_selection.enabled: false` (기본값) 이면 건너뛴다.
3. **모형 적합** — LGBM / XGBoost / CatBoost / ElasticNet 4종에 대해 (a) Optuna TPE 로 하이퍼파라미터 베이지안 최적화 → (b) StratifiedKFold OOF 평가 → (c) 최종 fit (`final_fit_strategy`) → (d) 테스트 데이터로 평가. 개별 모델 학습 완료 후 `ensemble` 설정에 따라 앙상블을 추가 생성한다 (기본 strategy 는 ElasticNet 이 켜져 있어야 동작 — "앙상블" 절 참고). primary_metric (기본 ROC-AUC) 기준으로 best 모델(앙상블 포함)을 선정한다.
4. **보고서 산출** — HTML / PDF 동일 내용 (Jinja2 + WeasyPrint). 모델 비교표, CV(OOF) 비교, **오버핏 점검 (Train vs Holdout, Δ)**, 튜닝 결과, ROC / PR 곡선, feature importance, score 분포, confusion matrix 포함.
5. **주기 스코어링** — 단일 artifact (preprocessor + model + metadata) 파일 1개 + 설정 YAML 1개로 운영 가능. cron 등에서 `python inference/run_all.py` 호출.

입력 데이터

- 학습 / 평가 데이터를 **별도 Parquet 파일** 로 받는다 (`train_data_path` , `test_data_path`). 내부에서 임의 분할하지 않는다.
- 학습/스코어링에 사용할 컬럼은 **CSV 파일** 로 명시한다. YAML 에서 `features_csv` 에 경로를 지정하면 `load_config` 가 자동으로 읽어 `cfg.features` 를 채운다.

CSV 형식 (필수 컬럼: `name` , `type` , `used`):

```
name,type,used
age,continuous,true
gender,category,true
income,continuous,true
internal_score,continuous,false
```

- `type`
- `continuous` — 연속형 (int / float). 결측·이상치·skew 변환·스케일링 적용.
- `category` — 범주형 (문자열 / 코드). 전처리에서 값 정규화 + 결측 대체, 인코딩은 모델 래퍼에서 처리.
- `used`
- `true` 인 행만 학습/스코어링에 사용된다 (`false` /공백 → 제외).
- 운영 중 컬럼을 켜고 끌 때 코드 변경 없이 본 CSV 만 수정하면 된다.

```
features_csv: ./configs/features.csv
```

결측치 처리

전처리 1단계. 학습 데이터에서 컬럼별 채움 값을 계산해 저장하고, 스코어링 시 동일하게 적용한다. 학습/스코어링 일관성을 보장한다.

```
preprocessing:
  numeric_null_strategy: median               # median | mean | constant
  numeric_null_fill_value: 0.0                # constant 일 때만 사용
  categorical_null_strategy: most_frequent    # most_frequent | constant
  categorical_null_fill_value: MISSING        # constant 일 때만 사용
```

- 수치형 전략: `median` (이상치에 강건, 기본) / `mean` / `constant`.
- 범주형 전략: `most_frequent` (기본) / `constant`.
- **전부-NaN 가드**: 학습 데이터에서 어떤 수치형 컬럼이 전부 NaN 이면 `median`/`mean` 으로 채움 값을 계산할 수 없어 silent NaN 전파를 막기 위해 명시적 `ValueError` 로 실패한다. `features.csv` 에서 해당 컬럼의 `used` 를 false 로 두거나 `constant` 전략으로 전환.
- 별도로 `PreprocessingPipeline.fill_missing_columns(df)` 가 스코어링 시 **컬럼 자체가 통째로 누락**된 케이스에 대비해 학습 시 산출한 기본값 (수치형 median, 범주형 최빈값) 으로 채워준다. 부분 NaN 은 imputer 가 처리.

범주형 값 정규화

결측 대체 전에 범주형 값을 같은 규칙의 문자열로 맞춘다 (`auto_ml/utils/categorical.py`). `1` / `1.0` / `"1"` 은 모두 `"1"`, `1.5` 는 `"1.5"`, bool 은 `"True"` / `"False"`, 문자열은 그대로 둔다. 학습 데이터에 결측이 있어 정수 코드 컬럼이 float 로 읽히고 추론 데이터가 int 로 들어오는 경우에도 같은 범주로 인식된다.

학습 때 본 범주 목록을 전처리기에 보관하고, 추론·SHAP·test 데이터에서 학습 때 없던 범주가 나오면 컬럼별 비율을 로그로 남긴다 (1% 이상 WARNING, 미만 INFO).

이 정규화 이전에 학습된 artifact 는 기존 방식(정규화 없음, XGBoost 는 LabelEncoder) 그대로 동작하므로 재학습 전까지 결과가 바뀌지 않는다.

이상치 처리 (백분위수 윈저라이징)

전처리 2단계. 학습 데이터의 컬럼별 분위수 `(quantile(lower_q), quantile(upper_q))` 를 경계로 사용해 윈저라이징하고, 스코어링 시 학습 시 저장한 동일 경계를 그대로 적용한다.

```
preprocessing:
  outlier_method: percentile     # percentile | none
  outlier_lower_quantile: 0.01
  outlier_upper_quantile: 0.99
  outlier_action: clip           # clip | null_then_impute
```

- `outlier_method`
- `percentile` — 학습 분포의 (lower_q, upper_q) 분위수를 경계로 사용. (기본)
- `none` — 비활성.
- `outlier_action`
- `clip` — 경계 밖 값을 경계로 잘라낸다. (기본)

- `null_then_impute` — 경계 밖을 NaN 으로 만든 뒤 학습 시 median 으로 다시 채운다.
- `outlier_lower_quantile < outlier_upper_quantile` 이고 둘 다 `[0, 1]` 범위여야 한다 (위반 시 `ValueError`).

Skew 변환

전처리 2.5단계. 학습 데이터의 컬럼별 skew 를 측정해 임계값 초과인 변수만 자동으로 선택해 분포를 대칭화한다. 학습 시 추정된 변환 파라미터를 저장해 스코어링에 동일 적용한다.

```
preprocessing:
  skew_method: signed_log1p      # signed_log1p | quantile_normal | none
  skew_threshold: 1.0            # |skew| > threshold 인 컬럼만 변환
```

- `skew_method`
- `signed_log1p` — `sign(x) * log1p(|x|)`. 음수까지 단조 보존, 무상태. (기본)
- `quantile_normal` — sklearn `QuantileTransformer` 로 학습 분포의 분위수를 표준정규로 매핑. 이상치/skew 모두에 강건. 학습 시 분위수 테이블을 저장.
- `none` — 비활성.
- 변환 대상은 학습 시 한 번만 결정된다. `|skew| > skew_threshold` 인 컬럼만 선택되며, 미선택 컬럼은 스코어링에서도 그대로 통과한다 (분포 일관성).
- 부스팅 트리는 단조 변환에 본질적으로 강건하지만, 선형/신경망 확장이나 분포 가정을 쓰는 진단에는 효과가 있다.

스케일링

전처리 3단계 (마지막). sklearn 의 스케일러를 pandas 입출력으로 감싼 얇은 래퍼.

```
preprocessing:
  scaling_method: standard           # standard | minmax | robust | none
```

- `standard` — `StandardScaler` (평균 0, 분산 1). 기본.
- `minmax` — `MinMaxScaler` ([0, 1]).
- `robust` — `RobustScaler` (median, IQR — 이상치에 강건).
- `none` — 비활성 (passthrough).

부스팅 트리만 쓰면 의미가 작지만, 선형/신경망 확장을 가정해 옵션으로 둔다. 학습 시 fit 된 스케일러는 artifact 에 저장되어 스코어링 시 동일 변환이 적용된다.

범주값 정리 · 범주형 분할 방식

범주형 변수 중 의미 있는 값만 따로 보고 나머지는 묶을 때 쓴다. 기본값이면 지금 동작과 같다.

```
preprocessing:
  categorical_levels:
# 결측 처리 직후, 변수마다 keep 또는 min_count 중 하나
    region:  { keep: ["서울", "부산"] }  # 이 값만 남기고 나머지 → __other__
    channel: { min_count: 50 }        # 학습 데이터에서 50건 이상인 값만 남김
  categorical_other_label: __other__  # 묶인 값 이름 (추론 때 처음 보는 값도 여기로)

models:
  lgbm:     { categorical_encoding: onehot }   # native(기본) | onehot
  catboost: { categorical_encoding: onehot }
```

- `categorical_levels` — 남길 값 목록은 학습 때 정해져 모델 파일에 저장되므로 추론에도 같게 적용된다. 결측을 채운 값(`MISSING`·최빈값) 도 남길 값이 아니면 묶인다.
- `categorical_encoding: onehot` — 트리 모델이 범주 하나씩 "그 값이냐 아니냐" 로만 분할한다(원-핫과 같은 분할). LightGBM·XGBoost 는 `max_cat_to_onehot`, CatBoost 는 `one_hot_max_size` 를 학습 데이터 범주 수에 맞춰 넣는다. 그 모델의 모든 범주형 변수에 적용되고, `fixed_params` 에 직접 쓴 값이 우선한다. ElasticNet 은 원래 원-핫이라 무시한다.
- 변수 이름은 그대로라 변수 선택·중요도·SHAP 은 원래 변수 단위로 나온다. 상세는 USER_GUIDE §9 · §13.

변수 선택 (Stability Selection)

전처리 직후, 모델 학습 직전에 적용되는 선택적 단계이다. 단일 fit 한 번의 우연성을 제거하기 위해, 학습 데이터에서 임의 부분표본을 반복 추출해 **변수가 얼마나 일관되게 선택되는지**(selection probability) 를 측정하고, 임계값 이상으로 자주 선택된 변수만 최종 학습에 사용한다. Meinshausen & Bühlmann (2010) 의 절차를 따른다.

절차

1. **Stratified 부분표본 추출** — 학습 데이터에서 `subsample_ratio` 비율로 `n_subsamples` 번 부분표본을 뽑는다. 클래스 비율이 부분표본마다 유지된다 (sklearn `StratifiedShuffleSplit`).
2. **부분표본별 변수 선택** — `base_estimator` 로 부분표본 한 번당 변수 셋을 뽑는다 (자세한 동작은 아래).
3. **선택 빈도 누적** — 각 변수가 부분표본 몇 번에서 뽑혔는지 세서 빈도 (0.0 ~ 1.0) 를 계산한다.
4. **임계값 이상 채택** — `frequency ≥ threshold` 인 변수만 최종 채택. 부족하면 `min_selected` fallback 으로 frequency 상위 N개를 보충한다.

Base estimator

```
feature_selection:
  enabled: true
  base_estimator: lasso           # lasso | lgbm
  n_subsamples: 200               # 부분표본 추출 횟수
  subsample_ratio: 0.5            # 각 부분표본 크기 비율 (Meinshausen 권고 = 0.5)
```

```
  threshold: 0.6                  # 채택 임계값 (보통 0.6~0.8)
  random_state: 42
  min_selected: 1                 # threshold 이상이 부족할 때 fallback 상위 N개

  # lasso 전용
  lasso_C: 0.1                    # L1 규제 강도 (낮을수록 더 sparse)

  # lgbm 전용
  lgbm_top_k: 30                  # 부분표본당 gain 상위 K개 채택
  lgbm_n_estimators: 100          # 부분표본 학습용 LGBM 트리 수 (가볍게)
  lgbm_learning_rate: 0.1
```

- **`lasso` (L1 로지스틱 회귀)** — 부분표본 단위 fit 후 **non-zero coefficient** 변수를 선택. 선형 신호에 강하고 빠르며 해석이 쉽다. 범주형 컬럼은 full-X 기준 **frequency encoding** 으로 1회 사전 변환해 부분표본마다 재인코딩하는 비용·bias 를 피한다.
- **`lgbm` (LightGBM gain 중요도)** — 부분표본 단위 fit 후 gain 중요도 **상위 `lgbm_top_k` 개**(중요도 > 0 만) 를 선택. 비선형·상호작용을 포착하지만 부분표본당 fit 비용이 크다. 범주형은 pandas `category` dtype 으로 LightGBM native 처리.

선택 기준이 다르므로 두 base estimator 가 항상 같은 변수를 뽑지 않는다. 선형 효과는 lasso 가, 상호작용·비선형은 lgbm 이 더 잘 찾는 경향이다.

설정 가이드

- **`n_subsamples`** — 표준 100~500. 소규모(< 5k 행)면 50~100 도 충분, 큰 데이터·고차원이면 200+. 추정 분산은 $\sqrt{n}$ 으로 줄어든다.
- **`subsample_ratio`** — 원 논문 권고 0.5. 부분표본이 너무 크면 모든 표본이 비슷해져 선택이 안정화되지 않고, 너무 작으면 fit 자체가 불안정해진다.
- **`threshold`** — 0.6 보수적, 0.8 매우 보수적. false-positive 통제 수준의 trade-off. 비교적 적은 채택을 원하면 0.7~0.8.
- **`min_selected`** — threshold 이상 변수가 부족할 때 frequency 상위 N개로 fallback. 모델 입력이 비어 학습이 실패하는 사고를 막는다. 운영 안정성용 안전망이며 평소엔 거의 발동되지 않아야 한다 (발동 시 WARN 로그).
- **`lasso_C`** — `LogisticRegression(C=...)` 의 역규제 강도. **낮을수록 규제 강함 → 더 sparse**. 0.1 이 시작점, feature 가 많이 살아 남으면 0.01 로 조이고, 너무 잘려나가면 1.0 으로 푼다.
- **`lgbm_top_k`** — 부분표본당 채택 상한. 일반적으로 전체 feature 의 30~50%. 너무 크면 거의 모든 feature 가 한 번씩 뽑혀 threshold 이상이 폭증한다.

Fallback / 실패 처리

- 부분표본 한 개 fit 이 실패해도 warning 만 남기고 다음 부분표본으로 진행 (예: 극단적인 stratify 결과로 한 클래스가 비는 경우).
- 모든 부분표본이 실패하면 `RuntimeError`. base_estimator 설정 점검 필요.

- threshold 이상 변수가 `min_selected` 미만이면 frequency 상위 N개로 fallback 하고 `fallback_used=True` 가 메타데이터에 기록된다.

산출물

`SelectionResult` 에 다음이 담긴다:

- `selected_features` — 채택된 변수 목록 (입력 컬럼 순서 유지).
- `frequencies` — 모든 변수의 선택 빈도 (0.0 ~ 1.0).
- `n_subsamples` — 실제 사용된 부분표본 수 (실패 제외).
- `fallback_used` — fallback 발동 여부.

선택 결과는 artifact 메타데이터에 저장되어 **스코어링 시 동일 컬럼 셋으로 강제 적용**된다. HTML/PDF 리포트에는 변수별 selection frequency 막대와 채택/제외 라벨이 표기된다 (리포트 8번 섹션 "변수 선택" 참고).

언제 켜고, 언제 끄나

- **켠다** — feature 수가 많고 일부가 noise 일 가능성이 있는 경우, 운영 안정성과 설명 가능성이 중요한 경우, 학습/스코어링 컬럼 셋을 명시적으로 줄이고 싶은 경우.
- **끈다 (`enabled: false`)** — 도메인 지식으로 이미 features.csv 가 정제됐고 feature 수가 적은 경우, 부스팅 트리 자체의 내장 selection 으로 충분하다고 판단되는 경우.

구현 메모

코드: `auto_ml/feature_selection/stability.py` 의 `StabilitySelector.fit_select(X, y) -> SelectionResult` 가 핵심 entry. 주요 흐름:

1. **base_estimator 별 사전 인코딩 1회** (`_encode_for_lasso` / `_prepare_for_lgbm`). lasso 는 범주형에 **full-X frequency encoding**, lgbm 은 pandas `category` dtype 으로 변환. 부분표본마다 재인코딩하지 않아 비용과 bias 를 줄인다.
2. **`StratifiedShuffleSplit(n_splits=n_subsamples, train_size=subsample_ratio, random_state=...)`** 으로 부분표본 인덱스 시퀀스를 생성. 각 인덱스에 대해 base 선택 함수 호출.
3. **부분표본 선택**:
4. `_lasso_select` — `LogisticRegression(solver="liblinear", C=lasso_C, max_iter=200)` 에 L1 규제 (sklearn 1.8 미만은 `penalty="l1"`, 1.8 이상은 `l1_ratio=1.0`). `abs(coef) > 1e-8` 인 컬럼을 채택.
5. `_lgbm_select` — `LGBMClassifier(...)` fit, gain importance 상위 `lgbm_top_k` 개 중 `importance > 0` 만 채택.
6. **실패 처리** — 부분표본 단일 실패는 try/except 로 잡아 warning 후 continue. 모든 부분표본이 실패하면 `RuntimeError`.
7. **threshold + fallback** — `frequencies[f] = count[f] / n_done`. `frequency ≥ threshold` 인 변수만 채택, 부족하면 빈도 상위 `min_selected` 개로 fallback 하고 `fallback_used=True` 가 메타에 기록된다.

모델 래퍼

`auto_ml/models/` 의 래퍼들은 모두 동일한 인터페이스 (`fit / predict_proba / feature_importance / shap_values`) 를 따른다.

모델	범주형 처리	early_stopping	Focal Loss
`LGBMModel`	pandas `category` dtype native	`callbacks=[early_stopping(rounds)]`	O
`XGBModel`	native (`enable_categorical=True` , 학습 범주로 고정한 `category` dtype, unseen → 결측)	생성자 인자 `early_stopping_rounds` (XGB 2.x sklearn API)	O
`CatBoostModel`	native (`cat_features` 인덱스 전달)	학습 인자 `early_stopping_rounds`	O (`PythonUse`
`ElasticNetModel`	내부 `OneHotEncoder` (학습 시 fit → 스코어링 재사용, `handle_unknown="ignore"`)	해당 없음 (무시)	X
`EnsembleModel`	서브모델에 위임	해당 없음 (no-op)	X

모든 모델은 동일한 전처리 결과와 (활성 시) 변수 선택 채택 컬럼 셋을 공유한 채 학습된다. **테스트 데이터 `primary_metric` 점수가 가장 좋은 모델(앙상블 포함)**이 best 로 선정되어 artifact 에 저장된다.

Trainer 흐름

`auto_ml/models/trainer.py` 의 `Trainer` 가 모델별로 다음을 수행:

1. (옵션) `HyperparameterOptimizer` 가 Optuna TPE 로 `tuning.cv_folds` OOF 평균 `primary_metric` 을 최대화하는 파라미터 탐색.
2. 튜닝된(또는 `fixed_params` 만) 파라미터로 `training.cv_folds` StratifiedKFold OOF 평가. 각 fold 의 best_iter 평균이 리포트의 `best_iter (avg)` 로 표기됨.
3. 최종 모델 생성 — `training.final_fit_strategy` 에 따라 다르다. 기본 `early_stop_on_test` 는 학습 데이터 전체로 fit 하면서 테스트셋을 early-stopping 검증셋으로 쓴다 (자세한 내용은 "최종 모델 학습 전략" 절).
4. 테스트 데이터로 평가 → 모델별 holdout 점수 산출.

모든 개별 모델 학습 완료 후, `ensemble.enabled: true` 이고 활성화된 모델이 2개 이상이면 `ensemble.strategy` 에 따라 `EnsembleModel` 을 만들어 `results["ensemble"]` 로 추가한다 (조건이

안 맞으면 WARNING 후 생략 — "앙상블" 절 참고). OOF 예측은 서브모델 OOF 의 가중 평균(재학습 없음), test/train 예측은 `EnsembleModel.predict_proba()` 로 실시간 계산한다.

앙상블

개요

`ensemble.enabled: true` (기본값) 이고 활성 모델이 2개 이상이면 개별 모델 학습 완료 직후 `EnsembleModel` 을 만든다. 구성 방식은 `strategy` 로 고른다.

```
ensemble:
  enabled: true
  strategy: elasticnet_plus_best   # elasticnet_plus_best (기본) |
weighted_average
  elasticnet_weight: null          # elasticnet_plus_best 전용. null = 점수 비례 자
동
```

strategy	구성	가중치
`elasticnet_plus_best` (기본)	ElasticNet + 나머지 활성 모델 중 test `primary_metric` 1위 1개	`elasticnet_weight` 지정 시 그 값, null 이면 두 모델 점수 비례
`weighted_average`	활성 모델 전부	점수 비례: `weight[i] = score[i] / sum(score)`

주의 — YAML 에 `elasticnet` 블록이 없으면 앙상블이 만들어지지 않는다. 기본 strategy 인 `elasticnet_plus_best` 는 ElasticNet 이 켜져 있어야 하는데, ElasticNet 의 코드 기본값은 `enabled: false` 다. 이 경우 학습은 계속되고 `Ensemble skipped — ...` WARNING 만 남는다. 앙상블을 쓰려면 `models.elasticnet.enabled: true` 를 두거나 `strategy: weighted_average` 로 바꾼다. 저장소의 예시 설정(`configs/example.yaml` , `examples/credit/config.yaml` , `examples/sample/config.yaml`)은 모두 ElasticNet 을 켜 두었다.

스코어링 시 동작

`EnsembleModel` 은 모든 서브모델 인스턴스를 내부에 보관한다. `best.joblib` 에 저장될 때 cloudpickle 이 전체 객체 그래프(서브모델 포함)를 직렬화하므로, `Scorer.from_artifact("best.joblib")` 한 번 호출로 앙상블 스코어링이 완전히 동작한다 — 개별 서브 artifact 가 함께 있지 않아도 된다.

SHAP

앙상블 예측은 서브모델 **확률**의 가중 평균이므로, SHAP 도 서브모델마다 확률 도메인으로 바꾼 뒤 같은 가중치로 평균한다 (`EnsembleModel.shap_values_proba`). 그래서 `base_value + sum(shap_*)` 가 앙상블의 실제 예측 점수와 정확히 일치한다. `cv_bagging` 의 fold 평균 모델도 같은 방식이다.

(logit 도메인 SHAP 을 평균하는 `shap_values()` 도 남아 있지만, 그 합에 sigmoid 를 씌운 값은 확률 평균 예측과 다르므로 해석에는 쓰지 않는다.)

Feature importance

서브모델마다 중요도 단위가 다르다 (LightGBM/XGBoost gain 은 수천 이상, CatBoost 는 합 100, ElasticNet 은 |계수|). 그대로 가중합하면 단위가 큰 모델이 사실상 전부를 차지하므로, 서로 다른 종류가 섞인 앙상블은 각 서브모델 중요도를 합 1 인 비율로 바꾼 뒤 앙상블 가중치로 합산한다 (결과도 합 1 — 리포트 CSV 의 `importance_gain` 컬럼이 이 비율 값이 된다). 같은 종류만 있는 `cv_bagging` fold 앙상블은 원래 단위 (gain 등)의 평균을 그대로 쓴다.

ElasticNet 모델

sklearn `LogisticRegression(solver='saga')` 기반 L1+L2 혼합 규제 선형 모델. 부스팅 트리가 놓치는 단순 선형 신호를 포착하거나, 해석 가능한 계수(coefficient) 가 필요한 도메인에서 유용하다. 기본값은 `enabled: false` (opt-in).

```
models:
  elasticnet:
    enabled: true
    fixed_params:
      max_iter: 2000
    search_space:
      C:        { type: float, low: 1.0e-3, high: 10.0, log: true }
      l1_ratio: { type: float, low: 0.0,   high: 1.0 }
```

- **`C`** — 역규제 강도. 낮을수록 규제 강함 (더 sparse 계수).
- **`l1_ratio`** — L1/L2 혼합 비율. 0.0 = 순수 L2 (Ridge), 1.0 = 순수 L1 (Lasso).
- **범주형 처리** — 내부 `OneHotEncoder` 로 자동 처리. `handle_unknown="ignore"` 로 스코어링 시 미학습 범주를 0-벡터로 안전하게 처리한다.
- **SHAP** — `shap.LinearExplainer` 를 사용한다. OHE 확장된 SHAP 값을 원본 피처 공간으로 합산해 반환하므로 SHAP 해석(`auto_ml.explain`) 과 완전히 호환된다.
- **Focal Loss** — 미지원. `loss: logloss` (기본값) 만 사용 가능.
- **`early_stopping_rounds`** — 무시된다 (sklearn 에는 해당 개념이 없음).

손실 함수 (Focal Loss)

기본 손실은 라이브러리 native binary log-loss (`binary` / `binary:logistic` / `Logloss`) 이다. **클래스 불균형이 큰 데이터** (예: positive 비율 < 5%) 에서는 **Focal Loss** (Lin et al., 2017, ICCV) 가 잘 맞춘 쉬운 샘플의 손실을 `(1 - p_t)^gamma` 로 감쇠시켜 어려운 샘플에 학습 신호를 집중시킨다.

```
models:
  xgb:
    loss: focal                         # logloss (기본) | focal — 모델별 독립
    fixed_params:
      eval_metric: auc                  # AUC 는 rank-invariant → custom obj 와 호환
      tree_method: hist
      n_estimators: 2000
      focal_gamma: 2.0                  # easy-example 감쇠 지수 (기본 2.0)
      focal_alpha: 0.25                 # 양성 클래스 가중 (기본 0.25)
    search_space:
      # focal 하이퍼파라미터도 탐색 가능:
      # focal_gamma: { type: float, low: 0.5, high: 4.0 }
      # focal_alpha: { type: float, low: 0.1, high: 0.9 }
```

수식 (이진):

```
p = sigmoid(z),  p_t = p if y=1 else 1-p,  a_t = alpha if y=1 else 1-alpha
FL = -a_t * (1 - p_t)^gamma * log(p_t)
```

동작 / 호환성 메모

- `loss` 는 **모델별로 독립**. 한 모델만 focal 로 두고 나머지는 logloss 로 둘 수 있다.
- `ElasticNetModel` 과 `EnsembleModel` 은 Focal Loss 를 지원하지 않는다. 이 두 모델에는 `loss: logloss` (기본값) 만 지정해야 한다.
- `focal_gamma` / `focal_alpha` 는 `fixed_params` 에 두거나 `search_space` 로 탐색 가능 (예약 키 — 라이브러리 원본 파라미터에는 전달되지 않고 wrapper 가 pop).
- 내부 동작: wrapper 가 `objective` / `loss_function` 를 numpy 기반 callable (또는 CatBoost `PythonUserDefinedObjective` 클래스) 로 주입한다. LightGBM 의 경우 `predict_proba` 가 raw margin 을 반환하므로 wrapper 가 자체적으로 sigmoid 를 적용 (XGBoost / CatBoost 는 그대로 [0,1] 확률 반환). 외부 인터페이스 (`predict_proba` 가 1차원 [0,1] ndarray) 는 logloss 와 동일하다.
- 호환 가능한 `eval_metric` / `metric` 은 **순위 기반** 지표 (`auc` , `pr_auc` , `AUC`). 절대값 기반 지표 (`binary_logloss` , `Logloss` 문자열 등) 는 raw score 에 잘못 적용되어 의미가 깨지므로 사용하지 않는다.
- 폐쇄망 친화: 추가 의존성 없음. numpy + 기존 라이브러리만 사용. callable 은 모듈 최상위 함수 + `functools.partial` 패턴이라 joblib pickle 가능 — 스코어링 별도 프로세스에서 artifact 가 정상 복원된다.

구현: `auto_ml/models/losses.py` 에 `focal_grad_hess` , `lgb_focal_objective` , `xgb_focal_objective` , `CatBoostFocalObjective` 가 정의되어 있다. 세 wrapper 의 `_resolved_params()` 가 `self.loss == "focal"` 일 때 본 모듈의 객체를 주입한다.

하이퍼파라미터 최적화

각 모델 블록에 `fixed_params` (항상 적용) 와 `search_space` (Optuna 탐색 범위) 를 둔다. `tuning.enabled: true` 이고 모델별 `search_space` 가 비어있지 않으면 Optuna TPE 가 KFold OOF `primary_metric` 을 최대화하는 파라미터를 찾는다.

설정

```
training:
  cv_folds: 5                       # 최종 OOF 평가용 fold 수
  random_state: 42
  early_stopping_rounds: 50         # 부스팅 모델 조기종료 라운드 수
  primary_metric: roc_auc           # 모델 선택 / 튜닝 목적함수

tuning:
  enabled: true
  n_trials: 30                      # 모델당 시도 횟수
  timeout: null                     # 초 단위 wall-clock 상한 (null 이면 제한 없음)
  cv_folds: 3                       # 튜닝 단계 fold 수 (보통 training.cv_folds 보다
작게)
  random_state: 42

models:
  lgbm:
    enabled: true
    fixed_params: { objective: binary, metric: auc, verbose: -1, n_estimators:
2000 }
    search_space:
      learning_rate: { type: float, low: 0.01, high: 0.3, log: true }
      num_leaves:    { type: int,   low: 15,   high: 255 }
      reg_lambda:    { type: float, low: 1.0e-8, high: 10.0, log: true }
```

`search_space` 형식

- **`type: float`** — `low`, `high`, `log` (선택, 기본 false).
- **`type: int`** — `low`, `high`, `log` (선택), `step` (선택, 기본 1).
- **`type: categorical`** — `choices: [...]`.

`search_space` 가 비어있으면 해당 모델은 `fixed_params` 만으로 학습한다 (튜닝 스킵).

`primary_metric` 지원 목록

`auto_ml/reporting/metrics.py:SUPPORTED_METRICS` 가 단일 진실 출처: `roc_auc | pr_auc | accuracy | precision | recall | f1 | ks | lift`. 모두 **최대화** 방향이다. 도메인별 권장:

- `roc_auc` — 일반 (기본).
- `pr_auc` — positive 비율이 매우 낮은 불균형 데이터.
- `ks` — 신용평점 컨텍스트.
- `f1` / `precision` / `recall` — 임계값 0.5 기준이라 운영 임계값을 다르게 쓰는 경우엔 `roc_auc` / `pr_auc` 가 더 안전.

early_stopping

`training.early_stopping_rounds` 는 부스팅 모델 세 종류에 모두 적용되지만 내부 구현은 다르다 (위 모델 래퍼 표 참고). 튜닝·OOF 평가 단계의 fold 학습에는 항상 적용된다. 최종 fit 은 `training.final_fit_strategy` 에 따라 다르다 — 기본 `early_stop_on_test` 는 테스트셋을 검증셋으로 early stopping 을 적용하고, `iteration_capping` 은 적용하지 않으며(트리 수 고정), `cv_bagging` 은 재학습하지 않는다.

과적합 완화 가이드

리포트 §3 "오버핏 점검" 의 `Δ = Train − Test` 가 큰 경우 (예: 주요 지표 기준 `|Δ| ≥ 0.05`), 다음 4축의 정규화 레버를 `search_space` 에 노출해 Optuna 가 자동으로 강도를 찾도록 한다. 기본 `configs/example.yaml` 와 `examples/credit/config.yaml` 에 이미 추가되어 있어 그대로 학습하면 적용된다.

4축 정규화 레버

축	LGBM	XGBoost	CatBoost
L1 (가중치 sparse 화 → 변수 선택 효과)	`reg_alpha` (1e-8 ~ 10, log)	`reg_alpha` (1e-8 ~ 10, log)	— (CatBoost 는 L1 미지원)
L2 (가중치 크기 제약)	`reg_lambda` (1e-8 ~ 10, log)	`reg_lambda` (1e-8 ~ 10, log)	`l2_leaf_reg` (1 ~ 100, log)
분할 보수성 (이득 임계값)	`min_gain_to_split` (0 ~ 0.5)	`gamma` (0 ~ 5)	— (depth 로 대체)
모델 native 노이즈	`feature_fraction` / `bagging_fraction` (0.6 ~ 1.0)	`subsample` / `colsample_bytree` (0.6 ~ 1.0)	`random_strength` (0 ~ 10) + `bagging_temperature` (0 ~ 1)

이미 노출된 트리 복잡도 레버 (`num_leaves` , `max_depth` , `depth` , `min_data_in_leaf` , `min_child_weight`) 와 함께 동시에 탐색된다.

Δ 가 큰 경우 권장 대처 순서

1. **`tuning.n_trials` 를 30~50 으로 증가** — 위 레버 추가로 trial 당 탐색 효율이 떨어졌으므로 trial 수를 늘려 균형을 맞춘다. credit/config 기본 `n_trials=15` 는 학습 시간 절약용 — 정규화 효과를 제대로 평가하려면 30+ 필요.
2. **변수 수가 많으면 `feature_selection.enabled: true` 활성화** — feature 수가 200+ 이면 stability selection 으로 noise 변수를 사전 제거하는 것이 가중치 정규화보다 효과가 큰 경우가 많다 (특히 LGBM/XGB 에서 L1 의 변수 선택 효과를 더 직접적으로 강제).
3. **트리 복잡도 상한 축소** — `num_leaves` 상한 255 → 63, `max_depth` 상한 10 → 6 등으로 좁혀 모델 표현력 자체를 제한.
4. **`training.early_stopping_rounds` 를 30~50 으로 유지** — 너무 작으면 과보수 (best_iter 미달), 너무 크면 과적합 진입.

운영 메모

- CatBoost 는 L1 정규화를 지원하지 않으므로 `l2_leaf_reg` 범위를 log scale `1 ~ 100` 으로 넓혀 더 강한 규제까지 시도 가능. CatBoost 의 내부 기본은 3 이라 하한 1.0 은 유지 (그 미만은 의미 없음).
- `random_strength` (CatBoost) 는 분할점 점수에 가우시안 노이즈를 더하는 정규 화. 큰 값은 학습을 보수적으로 만들지만 수렴 속도도 늦춘다 → `iterations` / `early_stopping_rounds` 와 함께 균형.
- 본 가이드의 모든 항목은 코드 변경 없이 `models.<name>.search_space` YAML 편집만으로 즉시 적용된다. `tuning/space.py:sample_params` 가 임의 키를 처리.

최종 모델 학습 전략 (final_fit_strategy)

왜 필요한가

기본 동작(`early_stop_on_test`)은 **최종 모델 fit 의 early-stopping 검증셋으로 테스트셋을 사용**한다. 즉 최종 트리 수(`best_iter`)가 테스트셋에 맞춰 정해지고, best 모델 선정도 같은 테스트 점수로 이뤄진다. 그 결과 리포트 §3 의 holdout 점수와 `Δ = Train − Test` 가 **낙관적으로 편향**되어 실제 일반화 gap 을 과소평가한다.

이를 줄이기 위해, **학습 단계에서 테스트셋을 전혀 노출하지 않는** 두 가지 선택적 전략을 제공한다. 기본값은 현행 동작이라 지정하지 않으면 동작이 바뀌지 않는다.

세 가지 전략

전략	동작	테스트 셋 노출	권장 상황
`early_stop_on_test` (기본)	학습 전체로 fit 하되 테스트셋을 early-stopping valid 로 사용	O (학습 신호로 노출)	현행 호환. 단일 데이터 셋 일관성을 우선할 때
`iteration_capping`	CV fold 들의 `best_iter` 를 집계해 트리 수를 고정하고, early stopping 없이 train 전체로 재학습	X	리포트 §3 오버핏 Δ 가 큰데 정직한 트리 수로 재학습하고 싶을 때
`cv_bagging`	최종 재학습 없이 CV 의 K 개 fold 모델을 **균등 가중 앙상블**로 묶어 최종 모델로 사용	X	분산이 크고 안정성을 원할 때. 재학습 비용도 절약

두 신규 전략은 학습 단계에서 테스트셋을 보지 않으므로 Δ 가 정직해진다 — 보통 현행보다 약간 커 보일 수 있는데, 이는 편향이 제거된 정상적인 현상이다.

설정

```
training:
  final_fit_strategy: early_stop_on_test   # early_stop_on_test (기본) |
iteration_capping | cv_bagging
  iteration_cap_aggregation: mean          # iteration_capping 일 때: mean |
median
  iteration_cap_headroom: 1.0              # iteration_capping 일 때: capped =
ceil(agg * headroom)
```

동작 / 호환성 메모

- **모델별 iteration 키 차이 자동 처리** — `iteration_capping` 은 LGBM/XGBoost 의 `n_estimators`, CatBoost 의 `iterations` 를 모델 래퍼의 `ITER_PARAM_NAME` 으로 자동 식별해 고정한다.
- **ElasticNet 면제** — 부스팅이 아니라 고정할 트리 수가 없으므로 `iteration_capping` 에서는 조용히 일반 refit(테스트셋 미노출)으로 동작한다.
- **`cv_bagging` + 앙상블** — `ensemble.enabled: true` 와 함께 켜면, 각 모델이 이미 fold 평균(bagging)이 되므로 그 위의 점수 비례 앙상블은 **자동 비활성화**되고 WARN 로그가 남는다.
- **best_iter 미발동 fold** — early stopping 이 발동하지 않은 fold 는 `n_estimators` / `iterations` 설정값을 fallback 으로 사용해 집계가 0 방향으로 끌려가지 않게 한다.
- **best_iter 의 의미** — 세 부스팅 모델 모두 "best 지점까지의 트리 개수" 로 통일되어 있다 (라이브러리 원래 값은 LightGBM 만 개수, XGBoost·CatBoost 는 0부터 세는 번호). 이 통일 이전에 학습된 artifact 메타데이터·리포트의 XGBoost/CatBoost `best_iter` 는 1 작게 표시돼 있다.

- **artifact 자기기술** — 사용된 전략은 `artifacts/.../models/<name>.joblib` 메타데이터의 `extra.training_mode` 에 기록된다 (`iteration_capping` 은 `iteration_cap` , `cv_bagging` 은 `cv_bagging` 부가 정보 포함). `cv_bagging` 의 sub-artifact 는 K 개 fold 모델을 품은 `EnsembleModel` 1개라 그 자체로 스코어링 / SHAP 해석에 사용할 수 있다.

SHAP 해석

학습된 모형이 각 입력 행을 왜 그렇게 예측했는지 **건별·변수별 기여도** 를 산출한다. LGBM/XGB/CatBoost 는 native API (`pred_contrib` / `ShapValues`) 를 사용하고, `ElasticNetModel` 은 `shap.LinearExplainer` 를 사용한다. `EnsembleModel` 은 서브모델별 확률 도메인 SHAP 의 가중 평균을 쓴다 (위 "앙상블 > SHAP" 참고). 모든 모델의 `shap_values()` 는 `(n, n_features + 1)` 형태로 반환 — Explainer 와 완전 호환.

```
# 학습 후 같은 config 로 호출 (저장소 루트에서)
python -m auto_ml.explain.runner --config configs/example.yaml
```

출력 스키마 (Wide)

```
<id_columns> + shap_<feature_1> + shap_<feature_2> + ... + base_value + score
```

- **`base_value`** : 모델의 평균 예측 확률 (각 행 동일).
- **`shap_<feature>`** : 해당 변수가 평균 대비 예측을 얼마나 끌어올렸/내렸는지 (확률 도메인).
- **`score`** : 최종 예측 확률 (= `Scorer.score` 와 동일).
- **가법성 보장**: 모든 행에서 `base_value + sum(shap_*) == score` (수치 오차 ≤ 1e-12).

도메인 변환 공식

native API 는 raw-margin (logit) 도메인 SHAP 을 반환한다. 본 라이브러리는 다음과 같이 확률 도메인으로 변환하면서 가법성을 보존한다:

```
p_base = sigmoid(base_raw)
p_pred = sigmoid(base_raw + sum(feature_raw))
shap_prob[i] = feature_raw[i] / sum(feature_raw) * (p_pred - p_base)
```

비율은 raw 도메인 그대로 유지되고, 합산만 `(score - base_value)` 와 정확히 일치하도록 스케일링된다.

설정

```
explain:
  input_path:  ./data/score_input.parquet    # 스코어링 입력과 동일해도 무방
  output_path: ./artifacts/explanations/shap.parquet
```

```
  id_columns:
    - user_id                                  # ScoringConfig 와 동일한 우선순위
```

`id_columns` 가 비어 있으면 top-level `id_columns` → artifact metadata 순으로 fallback.

설치

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

패키지 설치(`pip install -e .`) 는 필요 없다. 의존 라이브러리만 설치하고 저장소의 `.py` 파일을 직접 실행한다.

- Python 3.10 이상 (검증: 3.11, 3.13).
- `requirements.txt` 는 `numpy<2.0` 으로 고정되어 있고, 이에 맞춰 `pyarrow<26` 으로 제한한다 (pyarrow 26 이상은 NumPy 2 가 필요해 Parquet 읽기가 실패한다).

WeasyPrint 는 시스템 폰트와 일부 라이브러리(libpango 등) 가 필요하다. 폐쇄망에서는 wheelhouse + 시스템 패키지를 함께 배포한다.

사용

모든 명령은 **저장소 루트에서** 실행한다 (YAML 의 상대 경로가 현재 디렉토리 기준).

실행 샘플

데이터 생성 → 학습 3단계 → 추론 3단계를 한 번에 돌려보는 샘플 (약 1분 이내, 산출물은 `artifacts/sample/`):

```
python examples/sample/run_sample.py            # --skip-data, --only train|
inference 옵션
```

설정은 `examples/sample/config.yaml` , 컬럼 정의는 `examples/sample/features.csv` .

단계별 실행

```
# 0) 더미 데이터 생성 (검증용)
python examples/make_dummy_data.py

# 1) 학습 — 전처리 → 학습 → 후처리
python train/preprocess.py  --config configs/example.yaml
python train/train.py       --config configs/example.yaml
```

```
python train/postprocess.py --config configs/example.yaml
#   또는 한 번에: python train/run_all.py --config configs/example.yaml
#   산출물:
#     artifacts/models/best.joblib, artifacts/models/models/<name>.joblib
#     artifacts/reports/report.html, report.pdf
#     artifacts/predictions/test_predictions.parquet  (id + score + prediction +
target)

# 2) 추론 — 전처리 → 예측 → 후처리
python inference/preprocess.py  --config configs/example.yaml
python inference/predict.py     --config configs/example.yaml
python inference/postprocess.py --config configs/example.yaml
#   또는 한 번에: python inference/run_all.py --config configs/example.yaml
#   산출물: artifacts/scores/scores.parquet  (id_columns + score + prediction)

# 3) SHAP 해석 — 산출물:
#    artifacts/explanations/shap.parquet
#    (id_columns + shap_<feature>... + base_value + score, 확률 도메인)
python -m auto_ml.explain.runner --config configs/example.yaml
```

단계 간 중간 산출물

단계 스크립트는 다음 단계의 입력을 `work_dir` 에 저장한다. 기본 경로는 `<artifact_dir 의 부모>/work/{train,inference}` 이고 `--work-dir` 로 바꿀 수 있다.

단계	저장 파일	내용
`train/preprocess.py`	`train_preprocessed.pkl.gz`	fit 된 전처리기, 변수 선택 결과, 전처리된 train/test
`train/train.py`	`train_result.pkl.gz`	모델별 학습 결과 (`TrainingResult`)
`inference/preprocess.py`	`inference_preprocessed.pkl.gz`	모델 입력 행렬, id 컬럼
`inference/predict.py`	`inference_predicted.pkl.gz`	예측 확률, id 컬럼

후처리 단계는 앞 단계 파일만 읽으므로, 예를 들어 리포트 설정만 바꿔 `train/postprocess.py` 를 재실행하면 재학습 없이 리포트와 artifact 를 다시 만든다. 단, 앞 단계 이후 데이터·전처리·모델 설정을 바꿨다면 해당 단계부터 다시 실행해야 한다.

Python 코드에서 호출할 때는 `auto_ml.stages.train` / `auto_ml.stages.inference` 의 `preprocess` / `train` (`predict`) / `postprocess` 함수, 또는 일괄 실행용 `AutoMLPipeline(cfg).run()` / `auto_ml.scoring.runner.run_scoring(cfg)` 를 사용한다 (저장소 루트가 `sys.path` 에 있어야 한다).

모형 후보 관리 · 실행 이력 · 롤백

폴더 구조

```
artifact_dir/
├── best.joblib                ← 운영 모델 (scoring/explain 이 읽는 유일한 파일)
├── deployments.jsonl          ← 배포 이력 (운영 모델이 바뀔 때마다 한 줄, 추가만 함)
├── models/                    ← 최신 실행의 후보 사본 (set_best 기본 대상)
│   ├── lgbm.joblib  xgb.joblib  catboost.joblib
│   ├── elasticnet.joblib      ← enabled: true 일 때
│   └── ensemble.joblib        ← 앙상블이 생성된 경우
└── runs/<YYYYMMDD_HHMMSS>/    ← 실행 스냅샷 (학습마다 하나, 쓴 뒤 수정하지 않음)
    ├── models/*.joblib
    ├── reports/               ← 그 실행의 리포트 사본
    ├── predictions/test_predictions.parquet
    └── run_info.json          ← 설정 전체, 변수 목록, 데이터 경로·행 수, 모델별 지표,
                                 라이브러리 버전, git 커밋(있으면)
```

각 후보 파일은 전처리기·모델·메타데이터를 담은 독립 번들이라 그 자체로 스코어링 / SHAP 해석에 사용할 수 있다. `ensemble.joblib` 은 서브모델을 모두 내부에 포함한다.

학습 → 배포 흐름

1. 후보 모델을 `runs/<run_id>/models/` 에 저장하고 `models/` 를 그 사본으로 통째로 교체한다 (이전 실행 후보가 섞이지 않는다).
2. test 예측·리포트를 기존 위치에 쓰고 실행 폴더에도 사본을 남긴다.
3. `versioning.auto_deploy: true` (기본) 이면 best 를 운영 모델로 배포한다. 실행이 끝까지 완료된 뒤에만 운영 모델이 바뀐다.
4. `versioning.keep_last_runs` (기본 5) 를 넘는 오래된 실행을 지운다. **현재 운영 실행과 직전 운영 실행은 개수와 무관하게 지우지 않는다** (롤백 대상 보호).

운영 모델 교체(학습 후 자동 배포, `set_best` , `rollback`) 는 모두 같은 절차를 거친다. - **시험 스코어링:** 대상 모델을 실제로 열고 `scoring.input_path` 의 앞 100행으로 점수를 계산해 본다. 열리지 않거나 점수가 이상하면 교체하지 않는다. 입력에 없는 변수가 있으면 WARNING. - **원자적 교체:** 임시 파일에 다 쓴 뒤 한 번에 바꾼다. 교체 중에 추론이 읽어도 깨진 파일을 보지 않는다. - **이력 기록:** `deployments.jsonl` 에 시각·동작 (train/promote/rollback/legacy) · 실행·모델·점수· 직전 운영 모델을 남긴다.

```
versioning:
  auto_deploy: true      # false 면 학습은 실행만 남기고, 배포는 set_best 로 한다
  keep_last_runs: 5      # 보관할 최근 실행 수 (운영·직전 운영 실행은 항상 보관)
```

기존 폴더 — `runs/` 가 없는 예전 구조의 `artifact_dir` 은 다음 학습 때 기존 `models/` · `best.joblib` · 리포트·test 예측을 `runs/legacy_<원래 학습 시각>/` 으로 담아 둔다. 그래서 첫 롤백이 예전 운영 모델로 돌아갈 수 있다.

학습 시점에 best 강제 지정

```
training:
  primary_metric: roc_auc
  best_model: lgbm        # null/생략 시 자동 선정. 지정 시 그 모형을 best 로.
                          # 가능한 값: lgbm | xgb | catboost | elasticnet |
ensemble
```

알 수 없는 모형 이름이면 학습이 명시적 `ValueError` 로 실패한다.

학습 후 운영 모델 교체 · 롤백 (재학습 없이)

```
# 최신 실행 후보 중 하나로 교체
python -m auto_ml.cli.set_best --config configs/example.yaml --model xgb

# 이전 실행의 모델로 교체
python -m auto_ml.cli.set_best --config configs/example.yaml --run
20261110_093000 --model lgbm

# 직전 운영 모델로 되돌리기 — 한 번 더 실행하면 원래대로 (현재 ↔ 직전 전환)
python -m auto_ml.cli.rollback --config configs/example.yaml

# 실행 목록(운영·직전 운영 표시)과 최근 배포 이력 보기
python -m auto_ml.cli.set_best --config configs/example.yaml --list
```

`best.joblib` 의 메타데이터는 배포한 후보 파일의 메타데이터와 같고(학습 시각·전략별 상세· `run_id` 포함), `extra` 에 배포 정보가 추가된다: - `best_selection` : `auto_by_metric` (자동) / `user_override` (`training.best_model` 지정) / `promoted` (set_best) / `rollback` - `deployed_at` - `promoted_at` / `promoted_from` : set_best·rollback 인 경우만. `promoted_from` 은 이전 운영 모델 이름

주의 - 실행마다 test 데이터·변수 구성이 다를 수 있다. 실행 간 점수를 단순 비교하지 말고 `run_info.json` 을 함께 확인한다. - 코드나 라이브러리를 업데이트한 뒤에는 오래된 실행의 모델이 열리지 않을 수 있다. 이때 롤백은 시험 스코어링 단계에서 거부되고, 운영 모델은 바뀌지 않는다. - 같은 `artifact_dir` 로 학습 두 개를 동시에 실행하는 것은 지원하지 않는다.

CPU 스레드 수 (`n_jobs`)

LightGBM·XGBoost·CatBoost 가 쓸 스레드 수를 설정 YAML 에서 정한다. 속도만 바뀌고 예측 결과는 같다.

```
training:
  n_jobs: null    # 학습·튜닝·변수 선택. null = 노드 전체 코어(라이브러리 기본), 양수 = 그
수, -1 = 전체, -2 = 하나 남김
scoring:
  n_jobs: null    # 추론·SHAP. null = 학습 때 값 그대로
```

- `null` · `-1` 은 서버(노드) 가 보여주는 전체 코어 수다. 컨테이너·팟의 CPU limit 은 반영되지 않으므로 그런 환경에서는 양의 정수로 적는다.
- 양수가 사용 가능한 CPU 수보다 크면 사용 가능한 CPU 수로 줄인다.
- 모델 `fixed_params` 에 스레드 인자(`num_threads` · `nthread` · `thread_count` 등) 를 쓰면 학습에서는 그 값이 우선한다. 추론 스레드는 `scoring.n_jobs` 로 정한다.

후보 모형의 상세 리포트

비-best 모형마다 별도 sub-report 가 자동 생성된다:

```
reporting.output_dir/sub/lgbm/report.html
reporting.output_dir/sub/lgbm/report.pdf
reporting.output_dir/sub/lgbm/feature_importance.csv
reporting.output_dir/sub/xgb/...
reporting.output_dir/sub/elasticnet/...
reporting.output_dir/sub/ensemble/...
```

각 sub-report 는 점수 표(현 모형 vs best), feature importance, decile/confusion/ score distribution, 변수 선택 결과, 학습 설정/튜닝 요약을 담는다. 변수 선택 CSV(`feature_selection.csv`) 는 모형 무관이라 메인 리포트에만 1개 둔다.

타이타닉 end-to-end 예시

실제 공개 데이터셋(Titanic, 891 rows) 으로 전체 파이프라인을 검증할 수 있다.

```
# 데이터 준비 (GitHub 에서 1회 다운로드 → data/titanic/
{train,test,score_input}.parquet)
python examples/titanic/prepare_data.py

# 학습 — 산출물: artifacts/titanic/{models,reports,logs}/...
python train/run_all.py --config examples/titanic/config.yaml

# 스코어링 — 산출물: artifacts/titanic/scores/scores.parquet
python inference/run_all.py --config examples/titanic/config.yaml
```

Optuna 10 trials × 4 모델(LGBM·XGB·CatBoost·ElasticNet) × 3 fold + 앙상블 설정으로 10~20초 정도(환경에 따라 다름)에 끝나며, best 모델(CatBoost) test ROC-AUC 는 약 0.85 이다 (2026-11 기준 재실행

0.8458). 폐쇄망에서는 `TITANIC_CSV` 환경변수에 사전 배포한 CSV 경로를 지정하면 `prepare_data.py` 가 네트워크 없이 동일 산출물을 만든다.

폐쇄망 이관

다음을 옮기면 된다 (스코어링 운영 기준):

1. 의존 라이브러리 wheel (`pip wheel -r requirements.txt -w wheelhouse/`)
2. 본 repo 자체 (`auto_ml/` , `train/` , `inference/` — 설치 불필요)
3. 학습 산출물 `artifacts/models/best.joblib`
4. 설정 YAML **과 그 YAML 의** `features_csv` **가 가리키는 CSV** — `load_config` 가 CSV 를 읽으므로 없으면 `FileNotFoundError` 로 실패한다.

설정

전체 옵션은 `configs/example.yaml` 참고. 모든 키는 `auto_ml/config.py` 의 dataclass 와 1:1 대응한다.

작업 로그

실행마다 stage 별 별도 로그 파일이 자동 생성된다. 단계 스크립트는 `train_preprocess` / `train_train` / `train_postprocess` / `inference_preprocess` / `inference_predict` / `inference_postprocess` , `run_all.py` 는 `train` / `inference` , Python API(`AutoMLPipeline.run` / `run_scoring`) 는 `train` / `score` prefix 를 쓴다.

```
artifacts/logs/
├── train_preprocess_20260425_143052.log
├── train_train_20260425_143110.log
├── train_postprocess_20260425_143502.log
├── inference_20260425_180001.log          ← inference/run_all.py
└── explain_20260425_181500.log
```

설정 (`configs/example.yaml` 의 `logging` 섹션):

```
logging:
  log_dir: ./artifacts/logs
  level: INFO          # DEBUG | INFO | WARNING | ERROR
  to_stdout: true
  to_file: true
```

콘솔 출력과 파일 출력은 동시에 활성화 가능하며, `to_file: false` 로 두면 파일은 만들지 않는다. 각 로그는 시작/종료 마커, 단계별 진행, 모델 튜닝 결과, 산출물 경로, 총 소요 시간을 포함한다.

운영 메모

- 타깃은 0/1 만 허용 (`utils/validation.py` 가 검증).
- best 선정은 테스트 데이터 점수 기준. 기본 학습 전략은 최종 fit 의 early-stopping 에도 테스트셋을 쓰므로 holdout/Δ 가 낙관적으로 편향될 수 있다 — `training.final_fit_strategy` 를 `iteration_capping` / `cv_bagging` 으로 바꾸면 학습 단계에서 테스트셋을 노출하지 않는다 (자세한 내용은 "## 최종 모델 학습 전략" 절 참고).
- 손실 함수 기본은 binary logloss. `models.<name>.loss: focal` 로 모델별 Focal Loss 전환 가능 (`focal_gamma=2.0` , `focal_alpha=0.25` 기본). 자세한 내용은 "## 손실 함수" 절 참고.
- 학습 시 사용한 features 정의가 artifact 메타데이터에 저장되어 스코어링 시 동일 컬럼 셋·동일 전처리·동일 모델로 처리된다. 스코어링 입력에 feature 컬럼이 통째로 없으면 에러가 아니라 학습 시 기본값(수치형 median, 범주형 최빈값)으로 채우고 WARNING 을 남긴다.
- 스코어링 결과 컬럼: `<id_columns> + score + prediction` .
- 학습 시 best 모델의 holdout 예측을 함께 내보낸다 — `<artifact_dir>/../predictions/test_predictions.parquet` , 스키마 `<id_columns> + score + prediction + <target_column>` . 외부 BI / 추가 진단용 (overfit 검증, 임계값 튜닝 등).
- 학습 데이터에서 어떤 수치형 컬럼이 전부 NaN 이면 결측·이상치 단계가 명시적 `ValueError` 로 실패한다 (silent NaN 전파 방지). `features.csv` 의 `used` 를 false 로 두거나 해당 단계를 비활성화/ `constant` 전략으로 전환.